

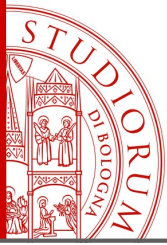
Ottimizzazione Matematica: Modelli

Daniele Vigo

D.E.I. – Università di Bologna

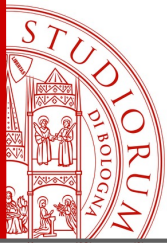
daniele.vigo@unibo.it

rev. 1.3 – 2026



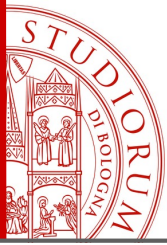
Dai problemi ai modelli

- Una volta individuato il problema, occorre classificarlo in modo da poter riconoscerlo come problema di un certo tipo...
...e quindi utilizzare algoritmi efficienti per risolverlo
- La classificazione è spesso favorita descrivendo il problema con un modello
 - favorisce l'astrazione e la precisione
 - permette di individuare similitudini tra problemi diversi
- Esistono diversi tipi di modelli:
 - in questo corso esamineremo principalmente modelli di programmazione lineare/intera

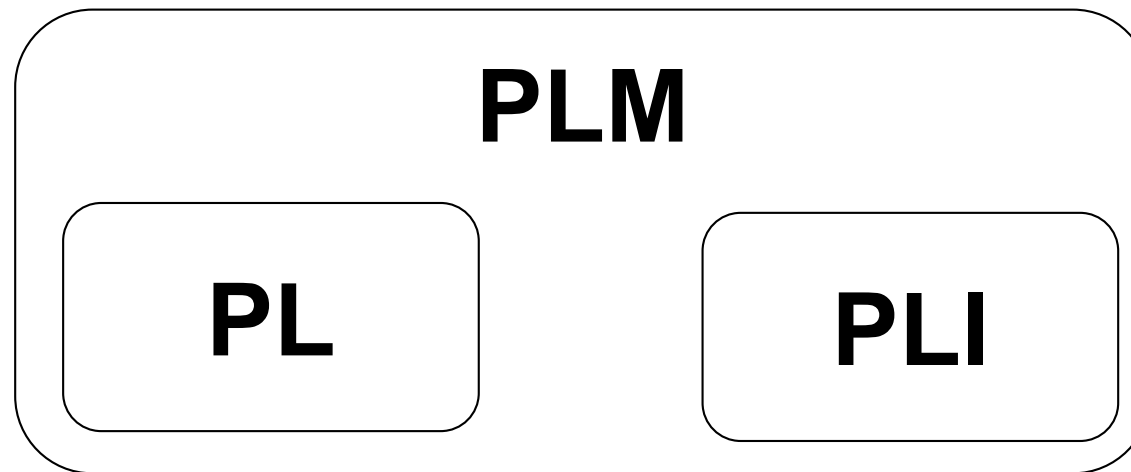


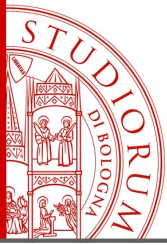
Programmazione Lineare

- Un problema di programmazione lineare (PL,LP) è un problema di ottimizzazione definito da:
 - Un numero finito $n \in N$ di **variabili decisionali** reali
$$x = (x_1, \dots, x_n) \in R^n$$
 - Una **funzione obiettivo** $f: R^n \rightarrow R$ nella forma $f(x) = cx$
 - Un insieme di m **vincoli lineari**, tutti in una delle forme seguenti: $ax = b, ax \leq b, ax \geq b$ con $a \in R^n, b \in R$
- Talvolta risulta indispensabile assumere che $x \in Z^n$, ovvero che le soluzioni ammissibili siano (vettori di) numeri interi.
- Si parla in questo caso di progr. lineare intera (PLI,ILP) o progr. lineare mista (PLM,MILP) se solo parte delle variabili sono intere



Programmazione lineare



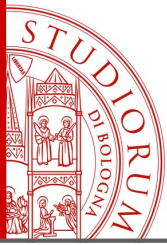


Programmazione lineare

- Un problema di PL può sempre essere espresso nella forma seguente:

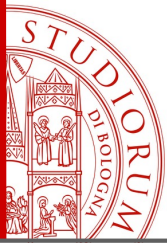
$$\min\{cx \mid Ax \leq b\}$$

- dove $A \in R^{m \times n}$ e $b \in R^m$, infatti:
 - Se il problema P è un problema di massimo, basta considerare $f(x) = -\min(-c)x$
 - Ogni vincolo $ax = b$ diventa la coppia di vincoli $ax \leq b$ e $ax \geq b$
 - Ogni vincolo $ax \geq b$ è equivalente a $(-a)x \geq (-b)$



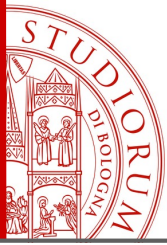
Tipo di variabili

- Variabili quantitative: reali
 - quantità di pezzi da produrre in un turno/ciclo
 - proporzioni o quantità di materie prime in una miscela
 - importi economici (costi, ricavi, ecc.)
 - istanti di tempo in cui avvengono eventi
 - coordinate in cui collocare oggetti ...
- Variabili quantitative: intere
 - quantità di pezzi/lotti da produrre in totale
 - numero di infrastrutture, veicoli, macchine da installare, acquistare, utilizzare
 - istanti di tempo (discretizzati) in cui avvengono eventi ...



Pianificazione della produzione

- Produzione di microprocessori.
 - due tipi: P e C
 - L'impianto è in grado di realizzare 3.000 "wafer" alla settimana: su ogni wafer trovano posto 500 C oppure 300 P.
 - La resa di un wafer dipende dal tipo di processore:
 - wafer C hanno una resa media del 60%,
 - wafer P hanno una resa media del 50%,
- I processori P si vendono a 500\$ al pezzo, mentre i C si vendono a 200\$ al pezzo.
- La divisione commerciale ha stabilito che la massima quantità di processori che possono essere messi sul mercato ogni settimana senza causare un calo dei prezzi è di 400.000 unità di P e di 700.000 unità di C.
- Si vuole determinare le quantità di ciascun tipo di processore da produrre settimanalmente in modo da massimizzare il ricavo totale.



Pianificazione della produzione

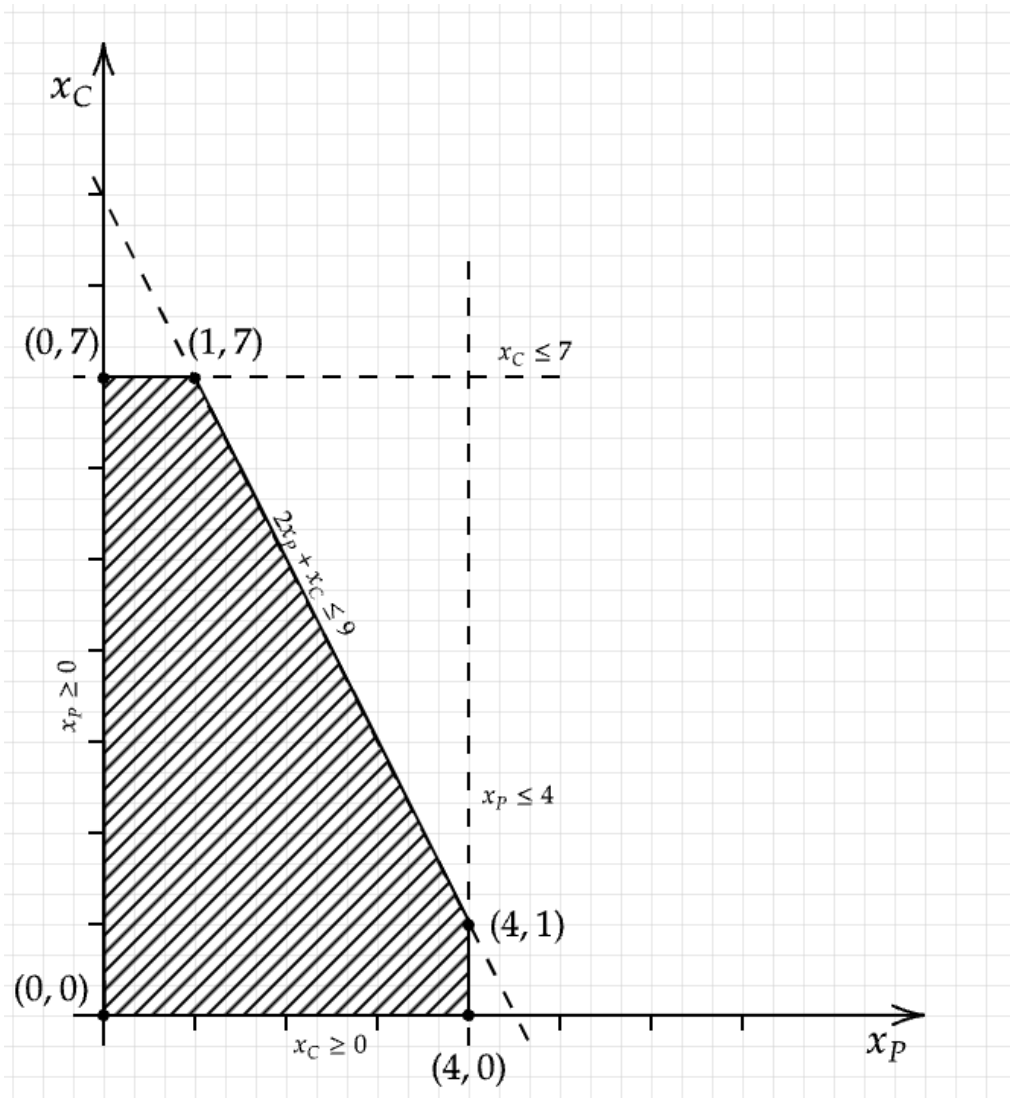
- x_P = n. di processori P prodotti settimanalmente
- x_C = n. di processori C prodotti settimanalmente

$$\begin{array}{rcll} \max & 500x_P & + & 200x_C \\ & x_P & & \leq 400.000 \\ & & & x_C \leq 700.000 \\ & 2x_P & + & x_C \leq 900.000 \\ & x_P & , & x_C \geq 0 \end{array}$$

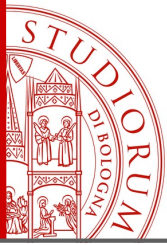
possiamo esprimere x_P e x_C in 10^5 pezzi. e la f.o. in 10^5€

$$\begin{array}{rcll} \max & 500x_P & + & 200x_C \\ & x_P & & \leq 4 \\ & & & x_C \leq 7 \\ & 2x_P & + & x_C \leq 9 \\ & x_P & , & x_C \geq 0 \end{array}$$

Soluzione grafica



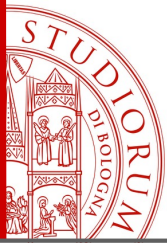
- La soluzione ottima coincide con il vertice $(4,1)$ ossia a
 $x_P = 400.000$,
 $x_C = 100.000$
- la soluzione non è intera rispetto ai wafer



Fonderia

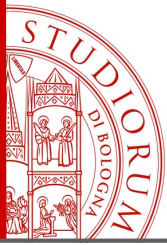
- Una fonderia deve produrre 1000 pezzi del peso ciascuno di un chilogrammo.
- Il ferro con cui tali pezzi sono fatti dovrà contenere manganese e silicio nelle seguenti quantità:
 - $0.45\% \leq \text{manganese (Mn)}$
 - $3.25\% \leq \text{silicio (Si)} \leq 5.5\%$
- Sono disponibili tre tipi di materiale ferroso con le seguenti caratteristiche:

Materiale	A	B	C
Si (%)	4.00	1.00	0.60
Mn (%)	0.45	0.50	0.40
Costo (€/Kg)	0.025	0.030	0.018



Fonderia

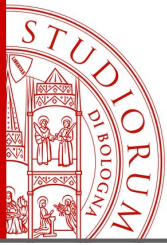
- Inoltre si può aggiungere Mn al costo di 10€/Kg
 - Determinare il piano di produzione che minimizza il costo del materiale utilizzato.
- ➔ Quantità di materiale per ciascuno dei tre tipi A, B, o C e di Mn puro da acquistare per produrre i 1000 pezzi richiesti, spendendo il meno possibile.



Fonderia

- x_1, x_2, x_3, x_4 = Kg di A,B,C,Mn da acquistare

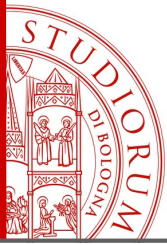
$$\begin{array}{llllllllll} \min & 0.025x_1 & + & 0.030x_2 & + & 0.018x_3 & + & 10x_4 & & \\ & 4x_1 & + & x_2 & + & 0.6x_3 & & & \geq & 3250 \\ & 4x_1 & + & x_2 & + & 0.6x_3 & & & \leq & 5500 \\ & 0.45x_1 & + & 0.5x_2 & + & 0.4x_3 & + & 100x_4 & \geq & 450 \\ & x_1 & + & x_2 & + & x_3 & + & x_4 & = & 1000 \\ & x_1 & , & x_2 & , & x_3 & , & x_4 & \geq & 0 \end{array}$$



Problema della dieta

... per cani e gatti!

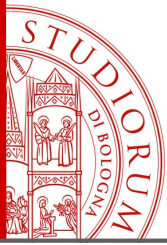
- n alimenti, m sostanze nutritive
- a_{ij} quantità della sostanza i in 1 unità dell'alimento j ($i = 1, \dots, m; j = 1, \dots, n$)
- d_i fabbisogno della sostanza i ($i = 1, \dots, m$)
- c_j costo 1 unità dell'alimento j ($j = 1, \dots, n$)
- Determinare il mix di alimenti di minimo costo che garantisce il fabbisogno delle sostanze nutritive



Problema della dieta

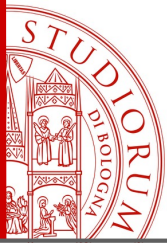
$x_j = \text{quantità alimento } j \text{ da acquistare } (j = 1, \dots, m)$

$$\begin{aligned} \min c(x) &= \sum_{j=1}^n c_j x_j \\ \text{(DP)} \quad \sum_{j=1}^n a_{ij} x_j &\geq d_i \quad i = 1, \dots, m \\ x_j &\geq 0 \quad j = 1, \dots, n \end{aligned}$$



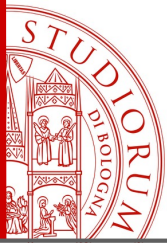
Tipo di variabili

- Variabili logiche: booleane (0,1)
 - esprimono decisioni/condizioni binarie (sì/no, falso/vero):
 - attivazione/spegnimento di macchine
 - acquisto/utilizzo o meno di una infrastruttura ...
 - spesso vengono usate come variabili ausiliarie per che stabiliscono il verificarsi o meno di condizioni specifiche:
 - utilizzo o meno di una risorsa
 - superamento di una soglia da parte di una v. quantitativa



Problema dello zaino (KP)

- Dato un contenitore (zaino) di capacità b e un insieme $E = \{1, 2, \dots, n\}$ di n oggetti, ciascuno caratterizzato da
 - peso a_i e profitto c_i , $i = 1, \dots, n$
- Determinare il sottoinsieme $S \subseteq E$ di oggetti di profitto massimo il cui peso complessivo non superi la capacità
- Noto come Knapsack Problem (KP) ha molte applicazioni pratiche:
 - caricamento di veicoli,
 - selezione di investimenti ...



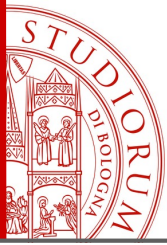
Problema dello zaino (KP)

- Il problema è dunque quello di determinare $S \subseteq E$ tale che:

- $$F = \{S \subseteq E : \sum_{i \in S} a_i \leq b\}$$

- $$c(S) = \sum_{i \in S} c_i$$

- le soluzioni possibili sono tutti i sottoinsiemi di E che sono $2^{|E|} = 2^n$
- la soluzione ottima è $S^* = \min\{c(S) : S \in F\}$
- è un tipico problema di selezione di elementi da un insieme

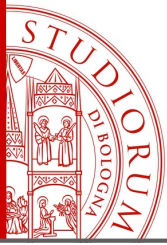


Problema dello zaino (KP)

- Problema di selezione:

$$\forall i = 1, \dots, n, \quad x_i = \begin{cases} 1 & \text{se } i \text{ selezionato} (\in S^*) \\ 0 & \text{altrimenti} \end{cases}$$

$$\begin{aligned} \text{(KP)} \quad & \max c(x) = \sum_{i=1}^n c_i x_i \\ & \sum_{i=1}^n a_i x_i \leq b \\ & x_i \in \{0,1\} \quad i = 1, \dots, n \end{aligned}$$

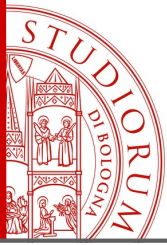


Esempio KP

- $n = 5$, capacità $b = 10$
- $c = (12, 12, 7, 6, 2)$, $a = (4, 5, 3, 3, 2)$
- Sol ammissibile $x = (1, 1, 0, 0, 0)$, $c(x) = 24$
- Un upper bound può essere ottenuto ordinando gli oggetti per rapporto c_i/a_i decrescente e inserendoli nello zaino fino al primo che non ci sta di cui si inserisce una frazione

$$U = [12 + 12 + 1/3 * 7] = 26$$

- Soluzione ottima $x^* = (1, 0, 1, 1, 0)$, $c(x^*) = 25$



Min-cost Spanning Tree

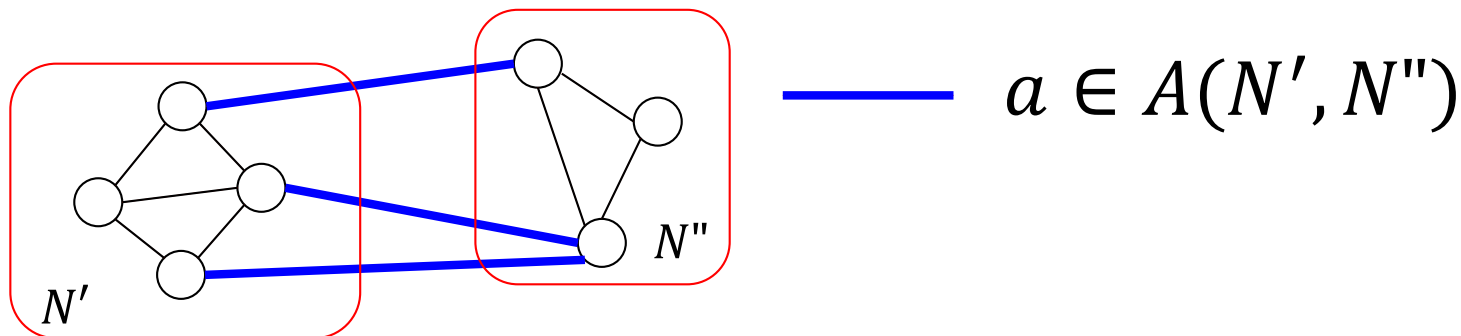
- Dato un grafo non orientato $G = (N, A)$
 - N insieme dei nodi (vertici) con $|N| = n$
 - A insieme degli archi con $|A| = m$
 - $\forall a \in A, c_a \geq 0$ costo dell'arco
- Determinare un grafo parziale $G' = (N, A')$ connesso e aciclico (spanning tree) di costo minimo
- $$F = \{T \subseteq A: (N, T) \text{ è una ST di } G\}$$
- $$c(T) = \sum_{a \in A} c_a$$
- la soluzione ottima è $T^* = \min\{c(T): T \in F\}$

Min-cost Spanning Tree

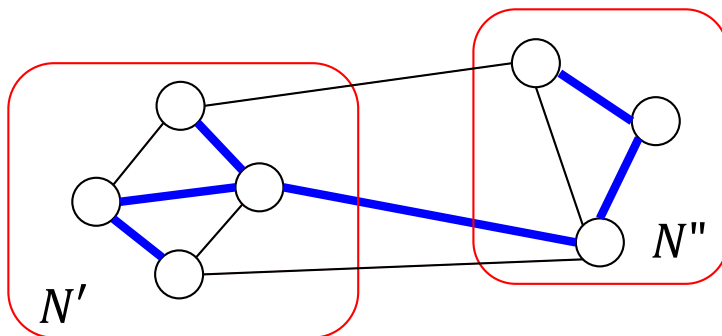
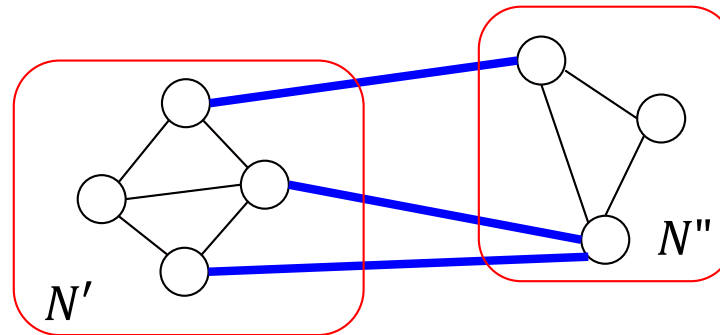
- T^* contiene un sottoinsieme di archi del grafo
→ problema di selezione

- $\forall a \in A, x_a = \begin{cases} 1 & \text{se } a \text{ selezionato} (\in T^*) \\ 0 & \text{altrimenti} \end{cases}$

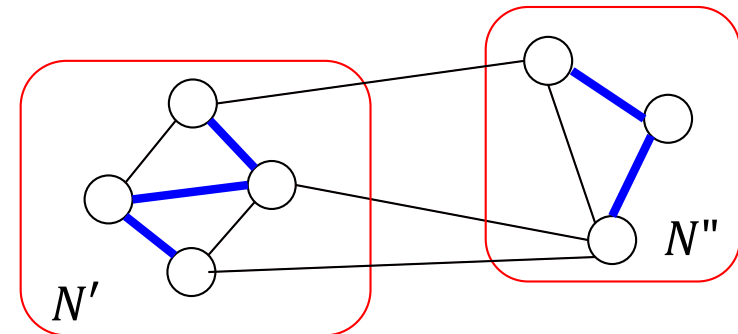
- Affinchè T sia connesso ogni taglio (N', N'') di G deve essere attraversato da un arco in T



Connessione

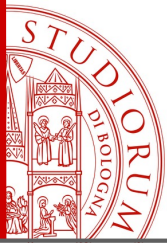


$T \in F$



$T \notin F$

- L'aciciclicità di T è implicata dalla minimizzazione di $c(T)$



Problema MST

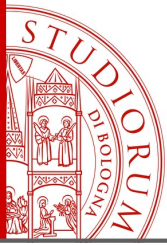
- Problema di selezione:

$$\forall a \in A, \quad x_a = \begin{cases} 1 & \text{se } a \text{ selezionato} (\in T^*) \\ 0 & \text{altrimenti} \end{cases}$$

$$\min c(x) = \sum_{a \in A} c_a x_a$$

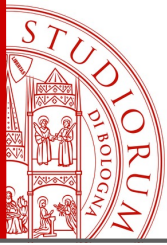
$$\begin{aligned} \text{(MST)} \quad & \sum_{a \in A(N', N'')} x_a \geq 1 \quad \forall (N', N'') \\ & x_a \in \{0, 1\} \quad a \in A \end{aligned}$$

- la formulazione ha un numero esponenziale di vincoli !



Problema del commesso viaggiatore

- Un commesso viaggiatore deve visitare n città in modo tale che il viaggio abbia lunghezza minima
- Dato un grafo orientato completo $G = (N, A)$
 - N insieme dei nodi (città) con $|N| = n$
 - A insieme degli archi con $|A| = m$
 - $\forall (i, j) \in A, c_{ij} \geq 0$ costo dell'arco
- Determinare un ciclo $G' = (N, A')$ di costo minimo
- ciclo Hamiltoniano: visita ciascun nodo una sola volta



TSP

- Traveling salesman problem (TSP)

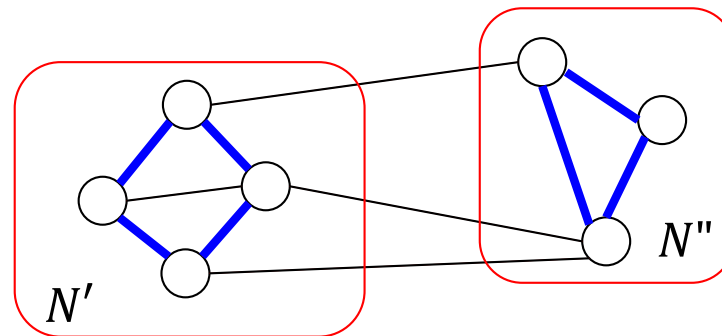
- $$F = \{P \subseteq A: (N, P) \text{ è ciclo Hamiltoniano di } G\}$$
$$c(P) = \sum_{(i,j) \in P} c_{ij}$$

- la soluzione ottima è $P^* = \min\{c(P): P \in F\}$
- P^* contiene un sottoinsieme di archi del grafo
→ problema di selezione

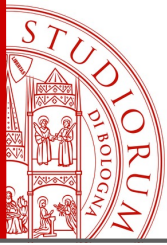
- $$\forall (i, j) \in A, \quad x_{ij} = \begin{cases} 1 & \text{se } (i, j) \text{ selezionato } (\in P^*) \\ 0 & \text{altrimenti} \end{cases}$$

Modello TSP

- dovendo visitare ogni nodo una sola volta P dovrà contenere esattamente due archi incidenti in ciascuna città
- $\sum_{j:(i,j) \in A} x_{ij} = 2 \quad i = 1, \dots, n$
- questi vincoli non sono però sufficienti ad imporre la connessione di P



- è necessario aggiungere vincoli come in MST



Modello TSP

$$\forall (i, j) \in A, \quad x_{ij} = \begin{cases} 1 & \text{se } (i, j) \text{ selezionato } (\in P^*) \\ 0 & \text{altrimenti} \end{cases}$$

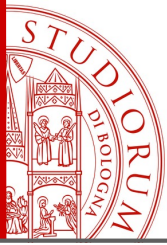
$$\min c(x) = \sum_{(i,j) \in A} c_{ij} x_{ij}$$

$$(TSP) \quad \sum_{j: (i,j) \in A} x_{ij} = 2 \quad i = 1, \dots, n$$

$$\sum_{(i,j) \in A(N', N'')} x_{ij} \geq 1 \quad \forall (N', N'')$$

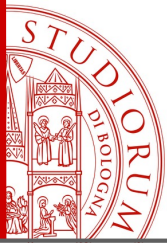
$$x_{ij} \in \{0, 1\} \quad (i, j) \in A$$

- la formulazione ha un numero esponenziale di vincoli !



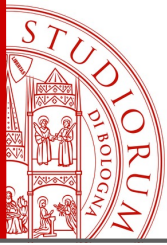
TSP e MST

- Si noti che la formulazione di TSP è la stessa di MST con l'aggiunta dei vincoli di grado, quindi $F(TSP) \subseteq F(MST)$
- la funzione obiettivo dei problemi è la stessa
→ MST è un rilassamento di TSP !
- per TSP e MST esistono diverse formulazioni alternative anche con un numero di vincoli polinomiale



Relazioni logiche tra variabili

- Spesso le relazioni intercorrenti tra le variabili logiche hanno esse stesse natura logica.
 - Ad esempio, x vale 1 se e solo se y e z valgono 1
- E' possibile modellare tutte le relazioni logiche tramite vincoli lineari
- **Negazione:** $(y = \neg x) \Rightarrow y = 1 - x$
- **Implicazione:** $x \rightarrow y \Rightarrow y \geq x$
 - se x vale 1, y è forzata a valere 1
 - attenzione: se x vale 0, y è libera !



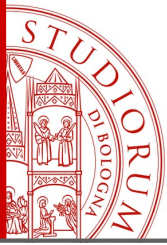
Relazioni logiche tra variabili

- **Intersezione, Congiunzione (AND):**

$$z = (x \wedge y) \rightarrow \begin{cases} z \leq x \\ z \leq y \\ z \geq x + y - 1 \end{cases}$$

- **Unione, Disgiunzione (OR):**

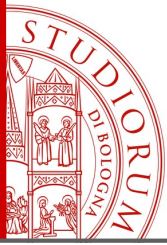
$$z = (x \vee y) \rightarrow \begin{cases} z \geq x \\ z \geq y \\ z \leq x + y \end{cases}$$



Vincoli di assegnamento

- Spesso si ha la necessità di modellare associazioni tra entità del problema
- Esempio: assegnazione di oggetti a luoghi
 - Un insieme $N = \{1, \dots, n\}$ di oggetti;
 - Un insieme $V = \{1, \dots, m\}$ di luoghi;
- vogliamo modellare le assegnazioni (i, j)

$$\forall(i, j), \quad x_{ij} = \begin{cases} 1 & \text{se oggetto } i \text{ assegnato a luogo } j \\ 0 & \text{altrimenti} \end{cases}$$



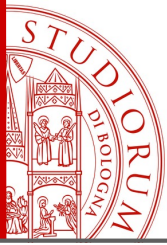
Vincoli di assegnamento

- Semi-assegnamento:
 - ogni oggetto può essere assegnato ad 1 solo luogo

$$\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n$$

- in alcuni casi non tutti i luoghi sono possibili per un oggetto
- $B_i \subseteq V$ insieme dei luoghi ammissibili per $i = 1, \dots, n$

$$\sum_{j \in B_i} x_{ij} = 1 \quad i = 1, \dots, n$$



Vincoli di assegnamento

- Assegnamento completo:
 - ogni oggetto può essere assegnato ad 1 solo luogo
 - ogni luogo un solo oggetto

- caso $m = n$

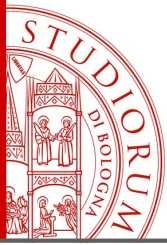
$$\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n \quad \sum_{i=1}^n x_{ij} = 1 \quad j = 1, \dots, m$$

- caso $m > n$ (luoghi non utilizzati):

$$\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n \quad \sum_{i=1}^n x_{ij} \leq 1 \quad j = 1, \dots, m$$

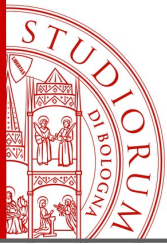
- caso $m < n$ (lavori non assegnati)

$$\sum_{j=1}^m x_{ij} \leq 1 \quad i = 1, \dots, n \quad \sum_{i=1}^n x_{ij} = 1 \quad j = 1, \dots, m$$



Problema dell'assegnamento

- Assegnazione di incarichi a persone
 - Un insieme $N = \{1, \dots, n\}$ di persone
 - Un insieme $V = \{1, \dots, n\}$ di incarichi
 - per ogni coppia persona-incarico (i, j)
 - c_{ij} = "costo" dell'assegnamento di i a j
 - ad esempio, tempo di esecuzione, se non possibile $c_{ij} = +\infty$
- Si vuole determinare l'assegnamento di costo complessivo minimo
 - tempo medio di esecuzione



Problema dell'assegnamento

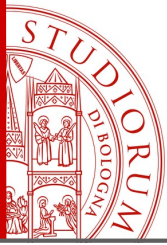
- $\forall(i, j), \quad x_{ij} = \begin{cases} 1 & \text{se persona } i \text{ assegnata a incarico } j \\ 0 & \text{altrimenti} \end{cases}$

$$\min c(x) = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

$$(AP) \quad \sum_{j=1}^n x_{ij} = 1 \quad i = 1, \dots, n$$

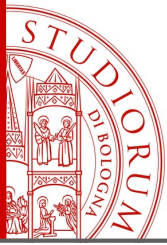
$$\sum_{i=1}^n x_{ij} = 1 \quad j = 1, \dots, n$$

$$x_{ij} \in \{0,1\} \quad i, j = 1, \dots, n$$



Ordinamento di lavori su macchine

- dati n lavori, ciascun lavoro $i = 1, \dots, n$:
 - disponibile dall'istante t_i (la lavorazione deve iniziare immediatamente)
 - supponiamo $t_1 \leq t_2 \leq \dots \leq t_n$
 - richiede di essere lavorato per d_i istanti,
 - termina a $t_i + d_i$
- m macchine identiche ($m \leq n$)
- Assegnare i lavori alle macchine senza sovrapposizione e minimizzando il numero di macchine usate



Assegnazione lavori

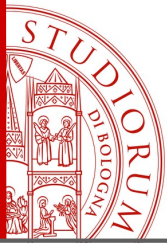
- La soluzione consiste in una partizione dei lavori in insiemi $N(1), \dots, N(m)$ eventualmente vuoti in cui $N(j)$ è l'insieme dei lavori sulla macchina j

$$\forall(i, j), \quad x_{ij} = \begin{cases} 1 & \text{se lavoro } i \text{ assegnato a macchina } j \\ 0 & \text{altrimenti} \end{cases}$$

$$i = 1, \dots, n; j = 1, \dots, m$$

- Ogni lavoro deve essere eseguito su una macchina

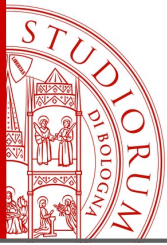
$$\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n$$



Incompatibilità lavori

- due lavori possono essere lavorati sulla stessa macchina se gli intervalli di lavorazione sono disgiunti
- $\forall h, k \quad t_h \geq t_k + d_k \quad \vee \quad t_k \geq t_h + d_h$
- se gli intervalli di lavorazione sono sovrapposti due lavori non possono essere eseguiti sulla stessa macchina
- $S(i)$ = lavori incompatibili con $i = 1, \dots, n - 1$
 $S(i) = \{h = i + 1, \dots, n: (]t_i, t_i + d_i, [\cap]t_h, t_h + d_h, [) \neq \emptyset\}$
- vincoli di compatibilità:

$$x_{ij} + x_{hj} \leq 1 \quad i = 1, \dots, n - 1; h \in S(i); j = 1, \dots, m$$



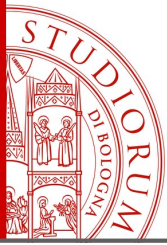
Obiettivo

- per "contare" le macchine usate abbiamo bisogno di variabili che ne indichino l'uso

- $\forall j = 1, \dots, m \quad y_j = \begin{cases} 1 & \text{se macchina } j \text{ usata} \\ 0 & \text{altrimenti} \end{cases}$

→ funzione obiettivo $\min \sum_{j=1}^m y_j$

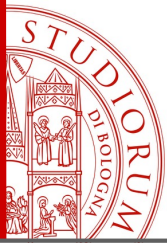
- legame tra lavori e macchine: $x_{ij} \rightarrow y_j$
- $x_{ij} \leq y_j \quad i = 1, \dots, n; j = 1, \dots, m$
- oppure $\sum_{i=1}^n x_{ij} \leq n y_j \quad j = 1, \dots, m$



Min Cardinality Machine Scheduling

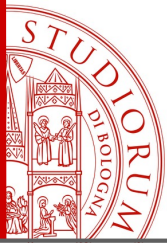
(MCMS)

$$\min c(y) = \sum_{j=1}^m y_j$$
$$\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n$$
$$x_{ij} + x_{hj} \leq 1 \quad \begin{array}{l} i = 1, \dots, n-1; \\ h \in S(i); j = 1, \dots, m \end{array}$$
$$x_{ij} \leq y_j \quad i = 1, \dots, n; j = 1, \dots, m$$
$$x_{ij} \in \{0,1\} \quad i = 1, \dots, n; j = 1, \dots, m$$
$$y_j \in \{0,1\} \quad j = 1, \dots, m$$



Selezione di sottoinsiemi

- Sia $N = \{1, \dots, n\}$ un insieme finito di elementi e sia $F = \{F_1, \dots, F_m\}$ una famiglia di suoi sottoinsiemi, dove $F_j \subseteq N$
- Ogni $F_j, j = 1, \dots, m$ ha un costo $c_j > 0$
- determinare un insieme di sottoinsiemi $D \subseteq F$ di costo minimo che soddisfi vincoli specifici
- Un generico F_j può essere rappresentato da un vettore binario di n elementi: $a_{ij} = 1$ se $i \in F_j$
- $F = A$ dove $a_{ij} = \begin{cases} 1 & \text{se } i \in F_j \\ 0 & \text{altrimenti} \end{cases}$



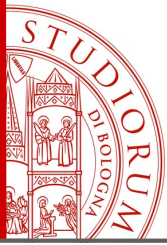
Selezione di sottoinsiemi

- La soluzione $D \subseteq F$ richiede di selezionare un sottoinsieme di F_j

→ variabili $x_j = \begin{cases} 1 & \text{se } j \in D \\ 0 & \text{altrimenti} \end{cases} \quad j = 1, \dots, m$

→ funzione obiettivo

$$\min \sum_{j=1}^m c_j x_j$$



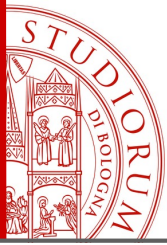
Set Covering Problem

- **Copertura** di N : ogni elemento $i \in N$ deve essere in **almeno** un sottoinsieme di D

$$\sum_{j=1}^m a_{ij}x_j \geq 1 \quad i = 1, \dots, n$$

$$\min c(x) = \sum_{j=1}^m c_j x_j$$

$$\begin{aligned} \text{(SCP)} \quad & \sum_{j=1}^m a_{ij}x_j \geq 1 & i = 1, \dots, n \\ & x_j \in \{0,1\} & j = 1, \dots, m \end{aligned}$$



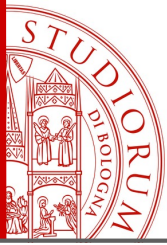
Set Partitioning Problem

- **Partizionamento** di N : ogni elemento $i \in N$ deve essere **esattamente** in un sottoinsieme di D

$$\sum_{j=1}^m a_{ij}x_j = 1 \quad i = 1, \dots, n$$

$$\min c(x) = \sum_{j=1}^m c_j x_j$$

$$\begin{aligned} \text{(SPP)} \quad & \sum_{j=1}^m a_{ij}x_j = 1 && i = 1, \dots, n \\ & x_j \in \{0,1\} && j = 1, \dots, m \end{aligned}$$



Set Packing Problem

- **Riempimento di N :** ogni elemento $i \in N$ deve essere **al più in** un sottoinsieme di D

$$\sum_{j=1}^m a_{ij}x_j \leq 1 \quad i = 1, \dots, n$$

- se problema di min: soluzione banale $D = \emptyset$

$$\max c(x) = \sum_{j=1}^m c_j x_j$$

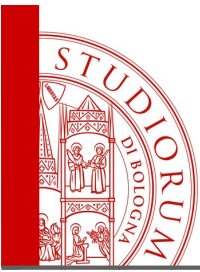
(SPkP)

$$\begin{aligned} \sum_{j=1}^m a_{ij}x_j &\leq 1 & i = 1, \dots, n \\ x_j &\in \{0,1\} & j = 1, \dots, m \end{aligned}$$

Localizzazione infrastrutture

- apertura centri CUP in una città divisa in 6 zone
- 1 sito per quartiere
- tempi di trasferimento tra i quartieri (in minuti):

	1	2	3	4	5	6
1	0	10	20	30	30	20
2	10	0	25	35	20	10
3	20	25	0	15	30	20
4	30	35	15	0	15	25
5	30	20	30	15	0	15
6	20	10	20	25	15	0



Localizzazione infrastrutture (2)

- massimo tempo di trasferimento 15 minuti

	1	2	3	4	5	6
1	0	10	—	—	—	—
2	10	0	—	—	—	10
3	—	—	0	15	—	—
4	—	—	15	0	15	—
5	—	—	—	15	0	15
6	—	10	—	—	15	0

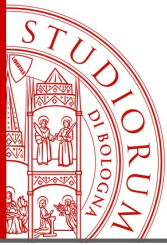
vicinanza zone



	1	2	3	4	5	6
1	1	1	0	0	0	0
2	1	1	0	0	0	1
3	0	0	1	1	0	0
4	0	0	1	1	1	0
5	0	0	0	1	1	1
6	0	1	0	0	1	1

A

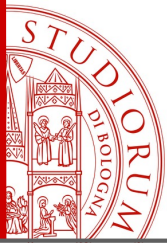
- *minimizzare il numero di centri aperti*



Modello matematico (PLI)

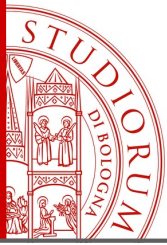
$$\bullet x_i = \begin{cases} 1 & \text{se si attiva il CUP nella zona } i \\ 0 & \text{altrimenti} \end{cases} \quad i = 1, \dots, n = 6$$

$$\begin{array}{llllllll} \text{min} & x_1 & + x_2 & + x_3 & + x_4 & + x_5 & + x_6 & \\ 1: & x_1 & + x_2 & & & & & \geq 1 \\ 2: & x_1 & + x_2 & & & & + x_6 & \geq 1 \\ 3: & & & + x_3 & + x_4 & & & \geq 1 \\ 4: & & & + x_3 & + x_4 & + x_5 & & \geq 1 \\ 5: & & & & + x_4 & + x_5 & + x_6 & \geq 1 \\ 6: & & + x_2 & & & + x_5 & + x_6 & \geq 1 \\ & x_1 & \dots & x_6 & \in \{0,1\} & & & \end{array}$$



Localizzazione infrastrutture nocive

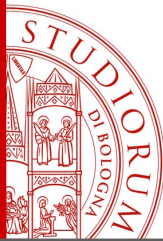
- Una regione deve localizzare dei termovalorizzatori nelle proprie provincie
- sono stati individuati m siti vicino alle n città principali
- per ogni sito $j = 1, \dots, n$:
 - b_j = costo di costruzione (10^6€)
 - c_j = capacità di trattamento (10^3t)
 - $a_{ij} = \begin{cases} 1 & \text{se città } i \text{ è vicina al sito } j \\ 0 & \text{altrimenti} \end{cases}$
- Sono disponibili B (10^6€) per la costruzione



Localizzazione infrastrutture nocive

- Ogni città non può avere più di un sito attivo "vicino"
- Si desidera massimizzare la capacità dei siti attivati nel rispetto del budget disponibile e del vincolo sulla vicinanza

$$x_i = \begin{cases} 1 & \text{se si attiva il sito nella zona } i \\ 0 & \text{altrimenti} \end{cases} \quad i = 1, \dots, n$$



Modello

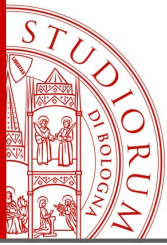
$$x_j = \begin{cases} 1 & \text{se si attiva il sito nella zona } j \\ 0 & \text{altrimenti} \end{cases} \quad j = 1, \dots, m$$

$$\max c(x) = \sum_{j=1}^m c_j x_j$$

$$(P) \quad \sum_{j=1}^m a_{ij} x_j \leq 1 \quad i = 1, \dots, n$$

$$\sum_{j=1}^m b_j x_j \leq B$$

$$x_j \in \{0,1\} \quad j = 1, \dots, m$$



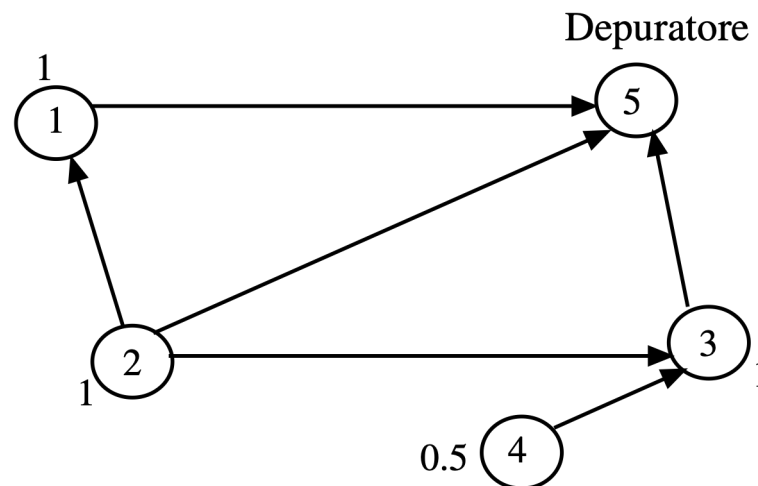
Variabili a valori discreti

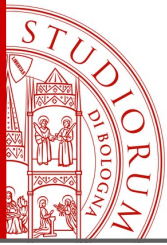
- Spesso le variabili sono vincolate ad assumere valori in un insieme che:
 - Non è semplicemente $\{0, 1\}$
 - Non è N
 - Non è un intervallo
- Ad esempio, x deve appartenere a un insieme $\{v_1, \dots, v_k\}$ di valori reali distinti
- introduciamo k variabili binarie $y_i \in \{0, 1\}, i = 1, \dots, k$

$$\sum_{i=1}^k y_i = 1 \qquad x = \sum_{i=1}^k v_i y_i$$

Progetto di reti

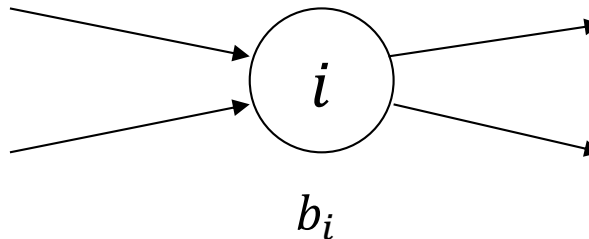
- si deve definire una rete fognaria per collegare alcuni centri abitati ad un depuratore
- ogni centro produce una quantità b_i di liquidi
- i collegamenti possibili sono descritti da un grafo orientato $G(V, A)$





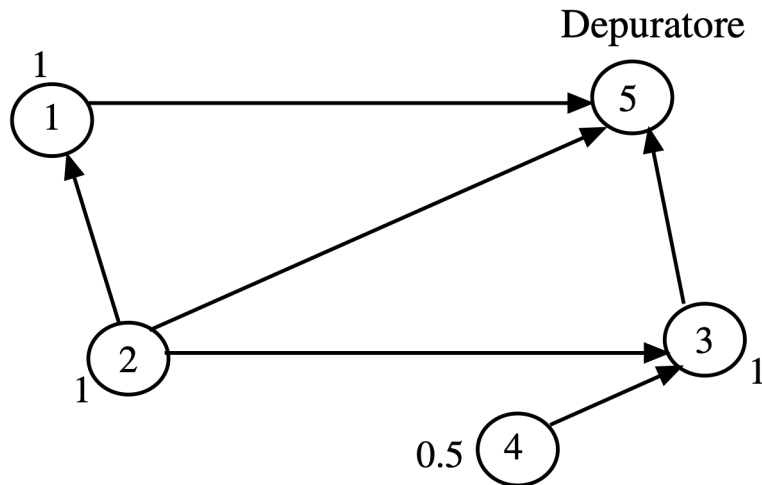
Progetto di reti

- Per ogni arco $(i, j) \in A$
 - x_{ij} = flusso di rifiuti sull'arco (=0 se non usato)
- Vincoli di bilanciamento del flusso:
 - per ogni nodo della rete il flusso portato dagli archi uscenti deve essere = al flusso entrante più la quantità prodotta nel nodo



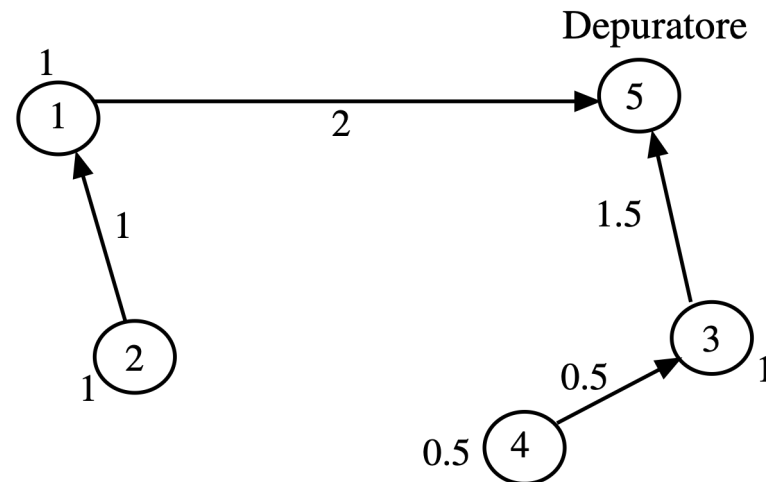
- Per il depuratore il flusso totale entrante è il totale della produzione dei centri

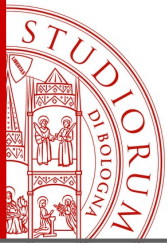
Bilanciamento del flusso



- $i = 1: x_{21} + 1 = x_{15}$
 - $i = 2: 1 = x_{21} + x_{23} + x_{25}$
 - $i = 3: x_{23} + x_{43} + 1 = x_{31}$
 - $i = 4: 0.5 = x_{43}$
- depuratore:
- $i = 5: x_{15} + x_{25} + x_{35} = 3.5$

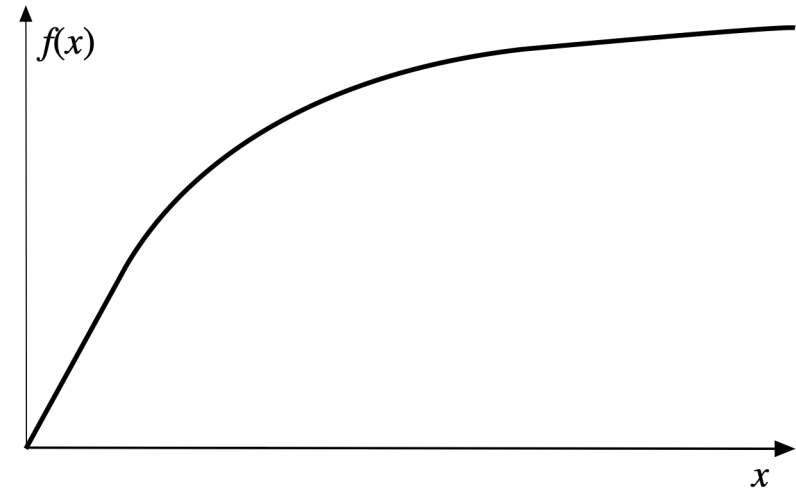
esempio di
soluzione
ammissibile





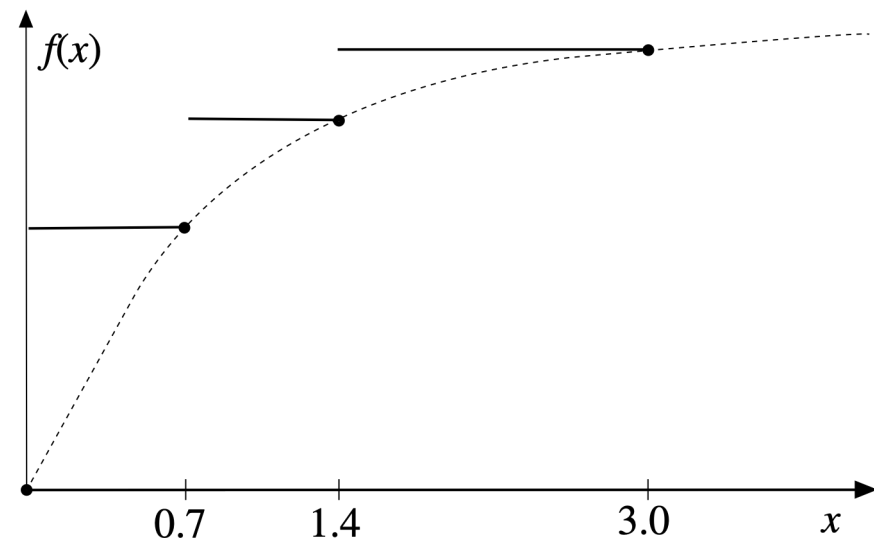
Funzione obiettivo

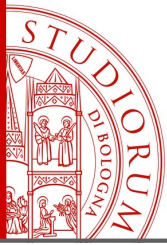
- Su ogni arco può essere installata una condotta la cui portata deve essere almeno pari al flusso che lo attraversa
- Il costo di posa è proporzionale alla portata secondo una funzione non lineare



Funzione obiettivo

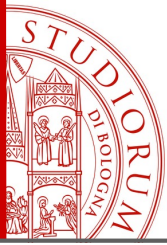
- In realtà le condotte sono disponibili solo per un sottoinsieme discreto di portate, ad esempio
- $p_h = \{0.0, 0.7, 1.4, 3\}$
- il costo di installazione è quindi una funzione a gradini
- si può ignorare la portata 0 $\Rightarrow p_h = \{0.7, 1.4, 3\}$





Modello

- Dobbiamo decidere quale portata installare su ogni arco della rete
 - minimizzando il costo di installazione
 - in modo che la portata sia compatibile ($\geq$) al flusso sull'arco ed i flussi siano ammissibili (=rispettino il bilanciamento)
- variabili ($\forall (i, j) \in A$)
 - x_{ij} = flusso sull'arco (i, j)
 - $y_{ij}^h = \begin{cases} 1 & \text{se sull'arco } (i, j) \text{ condotta di portata } p_h \\ 0 & \text{altrimenti} \end{cases}$
 - si può evitare di considerare la portata 0

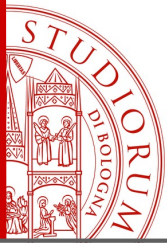


Modello

- Compatibilità tra x e y :

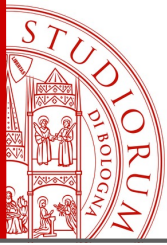
$$x_{ij} \leq 0.7y_{ij}^1 + 1.4y_{ij}^2 + 3y_{ij}^3$$
$$y_{ij}^1 + y_{ij}^2 + y_{ij}^3 \leq 1$$

- modello completo con $G = (V, A)$ e $|V| = n$
 - depuratore $i = n$,
 - produzioni $b_i \geq 0, i = 1, \dots, n - 1$ e $b_n = -\sum_{i=1}^{n-1} b_i$
 - k tipi di condotte con costo c_j e portata $p_j, j = 1, \dots, k$



Modello

$$\begin{aligned} & \min \sum_{(i,j) \in A} \sum_{h=1}^k c_h y_{ij}^h \\ & \sum_{l: (l,i) \in A} x_{li} - \sum_{j: (i,j) \in A} x_{ij} = b_i \quad i = 1, \dots, n \\ (P) \quad & x_{ij} \leq \sum_{h=1}^k p_h y_{ij}^h \quad (i,j) \in A \\ & \sum_{h=1}^k y_{ij}^h \leq 1 \quad (i,j) \in A \\ & x_{ij} \geq 0 \quad (i,j) \in A \\ & y_{ij}^h \in \{0,1\} \quad (i,j) \in A, h = 1, \dots, h \end{aligned}$$



Minima quantità positiva

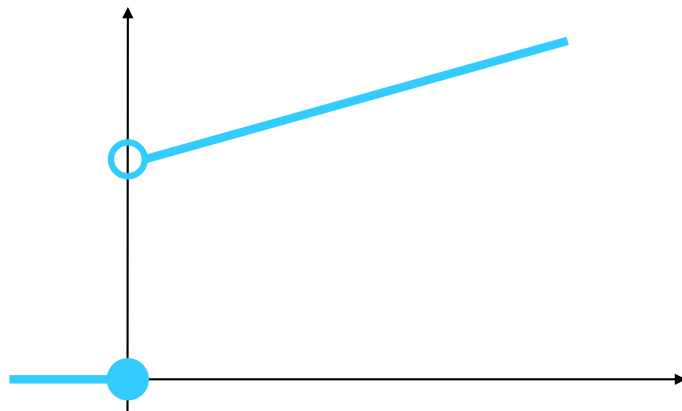
- Quando una variabile x rappresenta un certo livello di produzione, capita spesso che il valore di tale variabile debba appartenere a un insieme $\{0\} \cup [l, u]$
 - 0 rappresenta l'assenza di produzione, mentre
 - l'intervallo $[l, u]$ rappresenta i possibili livelli di produzione quando questa avviene
- Introduciamo una variabile logica $y \in \{0,1\}$ che indica la presenza o meno di produzione

$$ly \leq x \text{ e } x \leq uy$$

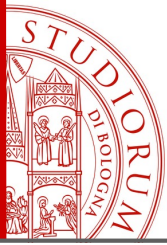
- se $y = 0$, allora $x = 0$. Altrimenti, $l \leq x \leq u$

Costo fisso di produzione

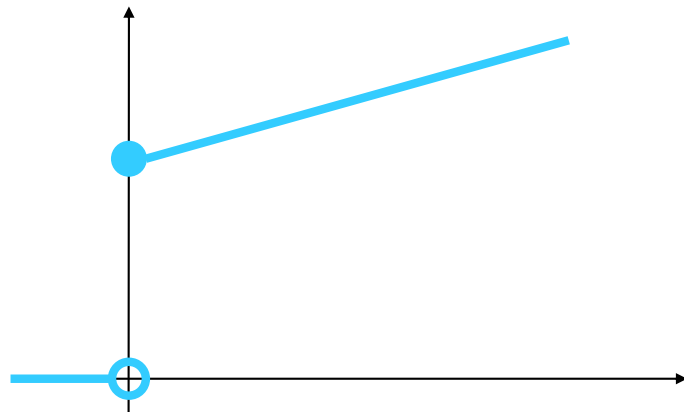
- Funzione con carico fisso
- Problema di mix di produzione
- Contributo prodotto j alla funzione obiettivo:
 - 0, se non prodotto ($x_j = 0$)
 - $F_j + p_j x_j$ se prodotto ($x_j > 0$)



- F_j = costo fisso (macchinari)
- p_j = costo di produzione
- discontinuità nell'origine



Costo fisso di produzione (2)



$$y_j = \begin{cases} 1 & \text{se } x_j > 0 \\ 0 & \text{se } x_j = 0 \end{cases}$$

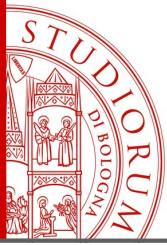
$$\min \sum_{j=1}^n F_j y_j + p_j x_j$$

$$Ax \geq b$$

$$x \geq 0$$

$$y \in \{0,1\}$$

- vincoli di tipo **logico** (*if ...*)
- occorre legare tra loro le x e le y con vincoli **lineari**



Costo fisso di produzione (3)

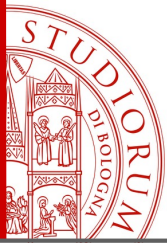
$$y_j = \begin{cases} 1 & \text{se } x_j > 0 \\ 0 & \text{se } x_j = 0 \end{cases} \quad \rightarrow \quad x_j \leq M y_j$$

con $M \gg 1 (\cong +\infty)$

- validità del vincolo:
 - se $x_j > 0$ y_j deve essere = 1 ($x_j \leq +\infty$)
 - se $x_j = 0$ y_j può essere = 0 o 1 ($0 \leq 0$ o $+\infty$)

oppure

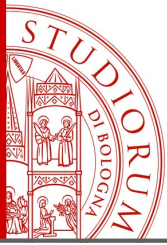
- se $y_j = 0$ x_j deve essere = 0 ($x_j \leq 0$)
 - se $y_j = 1$ x_j può essere = 0 o > 0 ($x_j \leq +\infty$)
- il vincolo impone solo parte della relazione logica



Costo fisso di produzione (4)

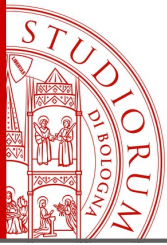
$$\begin{array}{ll} \min \sum_{j=1}^n F_j y_j + p_j x_j & \\ (P) \quad Ax \geq b & \text{vincoli produzione} \\ x_j \leq M y_j & j = 1, \dots, n \\ x_j \geq 0 & j = 1, \dots, n \\ y_j \in \{0,1\} & j = 1, \dots, n \end{array}$$

- non è necessario imporre anche l'altra metà della relazione logica
- una soluzione con $x_j = 0$ ed $y_j = 1$ non può essere ottima (ne esiste una equivalente che costa meno)



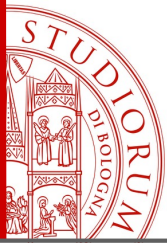
Variabili ausiliarie

- Variabili quantitative e logiche possono essere mischiate nei vincoli ma generalmente rimangono variabili decisionali
- A volte invece alcune variabili hanno un ruolo *ausiliario*: servono solo per esprimere linearmente una specifica condizione
 - ad es. le y_i del costo fisso



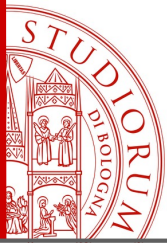
Vincoli di soglia

- Date n variabili $\{x_1, \dots, x_n\}$ può essere necessario definire dei valori soglia mediante vincoli lineari
- Es. $l \leq \min\{x_1, \dots, x_n\}$ o $u \geq \max\{x_1, \dots, x_n\}$
- si usano n vincoli lineari
- $l \leq x_i$ o $u \geq x_i \quad i = 1, \dots, n$
- Se in funzione obiettivo si massimizza l (o minimizza u), il valore ottimo coinciderà con il min (o max) valore di $\{x_1, \dots, x_n\}$



Ordinamento di lavori su macchine

- variante di MCMS
- Dati n lavori, ciascun lavoro $i = 1, \dots, n$:
 - richiede di essere lavorato per d_i istanti,
- m macchine identiche ($m \leq n$)
- Assegnare i lavori alle macchine senza sovrapposizione e minimizzando il tempo massimo di completamento dei lavori sulle macchine



Assegnazione lavori

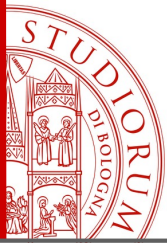
- La soluzione consiste in una partizione dei lavori in insiemi $N(1), \dots, N(m)$ eventualmente vuoti in cui $N(j)$ è l'insieme dei lavori sulla macchina j

$$\forall(i, j), \quad x_{ij} = \begin{cases} 1 & \text{se lavoro } i \text{ assegnato a macchina } j \\ 0 & \text{altrimenti} \end{cases}$$

$$i = 1, \dots, n; j = 1, \dots, m$$

- Ogni lavoro deve essere eseguito su una macchina

$$\sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n$$



Tempo di completamento

- Per ogni macchina il tempo di completamento t_j è la somma dei tempi dei lavori ad essa assegnati:

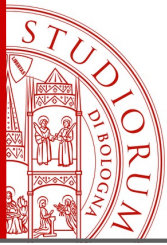
$$t_j = \sum_{i=1}^n d_i x_{ij} \quad j = 1, \dots, m$$

- Il tempo massimo di completamento è

$$t = \max\{t_j, j = 1, \dots, m\}$$

- che può essere definito da m vincoli lineari

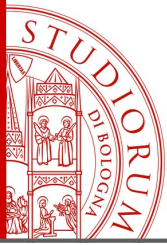
$$t \geq \sum_{i=1}^n d_i x_{ij} \quad j = 1, \dots, m$$



Min Makespan Machine Scheduling

- anche noto in letteratura come $P||C_{max}$

$$\begin{aligned} (MMMS) \quad & \min t \\ & \sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n \\ & \sum_{i=1}^n d_i x_{ij} \leq t \quad j = 1, \dots, m \\ & x_{ij} \in \{0,1\} \quad i = 1, \dots, n; j = 1, \dots, m \end{aligned}$$

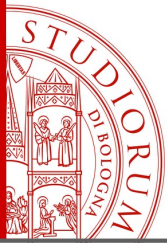


Valore Assoluto

- Se si deve rappresentare $|g(x)|$:
- **vincoli:** $|g(x)| \leq b$ può essere espresso dalla coppia di vincoli

$$g(x) \leq b \text{ e } -g(x) \leq b$$

- **funzione obiettivo:** $z = \max |f(x)|$ può essere calcolato risolvendo separatamente
- $z' = \max\{f(x), x \in F\}$ e $z'' = \max\{-f(x), x \in F\}$ e definendo $z = \max\{z', z''\}$
- analogamente per altri casi

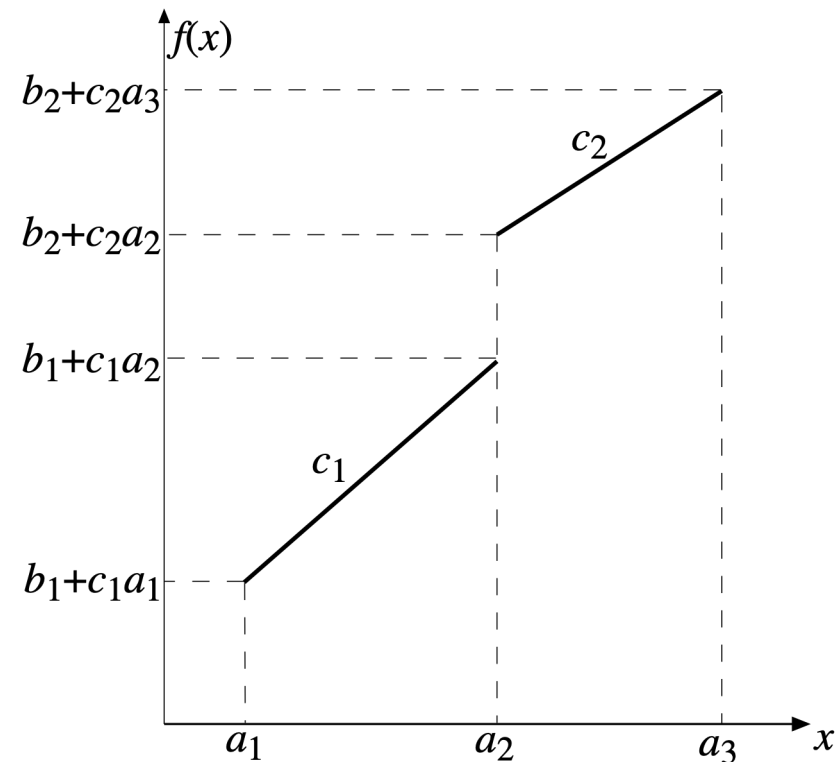


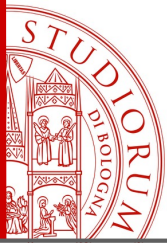
Funzioni lineari a tratti

- in molti casi è necessario usare funzioni lineari a tratti per descrivere vincoli o funzione obiettivo
- estensione del carico fisso

$$f(x) = \begin{cases} b_1 + c_1x & \text{se } x \in [a_1, a_2] \\ b_2 + c_2x & \text{se } x \in (a_2, a_3] \end{cases}$$

con $b_1 + c_1a_2 \leq b_2 + c_2a_2$





Funzioni lineari a tratti

- come nel carico fisso si possono usare variabili ausiliarie booleane:

$$y_1 = \begin{cases} 1 & \text{se } x \in [a_1, a_2] \\ 0 & \text{altrimenti} \end{cases} \quad \text{e} \quad y_2 = \begin{cases} 1 & \text{se } x \in (a_2, a_3] \\ 0 & \text{altrimenti} \end{cases}$$

- poichè x deve appartenere ad un solo intervallo

$$y_1 + y_2 = 1$$

- se solo 2 intervalli basta una sola variabile booleana y dato che $y_1 = y$ e $y_2 = 1 - y$ e il vincolo non è necessario

Funzioni lineari a tratti

- introduciamo anche 2 variabili ausiliarie quantitative:

$$z_1 = \begin{cases} x - a_1 & \text{se } x \in [a_1, a_2] \\ 0 & \text{altrimenti} \end{cases} \quad \text{e} \quad z_2 = \begin{cases} x - a_2 & \text{se } x \in (a_2, a_3] \\ 0 & \text{altrimenti} \end{cases}$$

- le variabili ausiliarie sono completamente definite dai vincoli

$$0 \leq z_1 \leq (a_2 - a_1)y_1$$

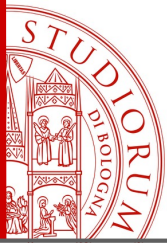
$$0 \leq z_2 \leq (a_3 - a_2)y_2$$

$$y_1 + y_2 = 1$$

$$y_1, y_2 \in \{0,1\}$$

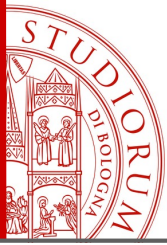
- $g(x)$ è esprimibile dalla funzione lineare $f: R^4 \rightarrow R$

$$\begin{aligned} f(z_1, z_2, y_1, y_2) &= b_1 y_1 + c_1(a_1 y_1 + z_1) + b_2 y_2 + c_2(a_2 y_2 + z_2) = \\ &= (b_1 + c_1 a_1) y_1 + c_1 z_1 + (b_2 + c_2 a_2) y_2 + c_2 z_2 \end{aligned}$$



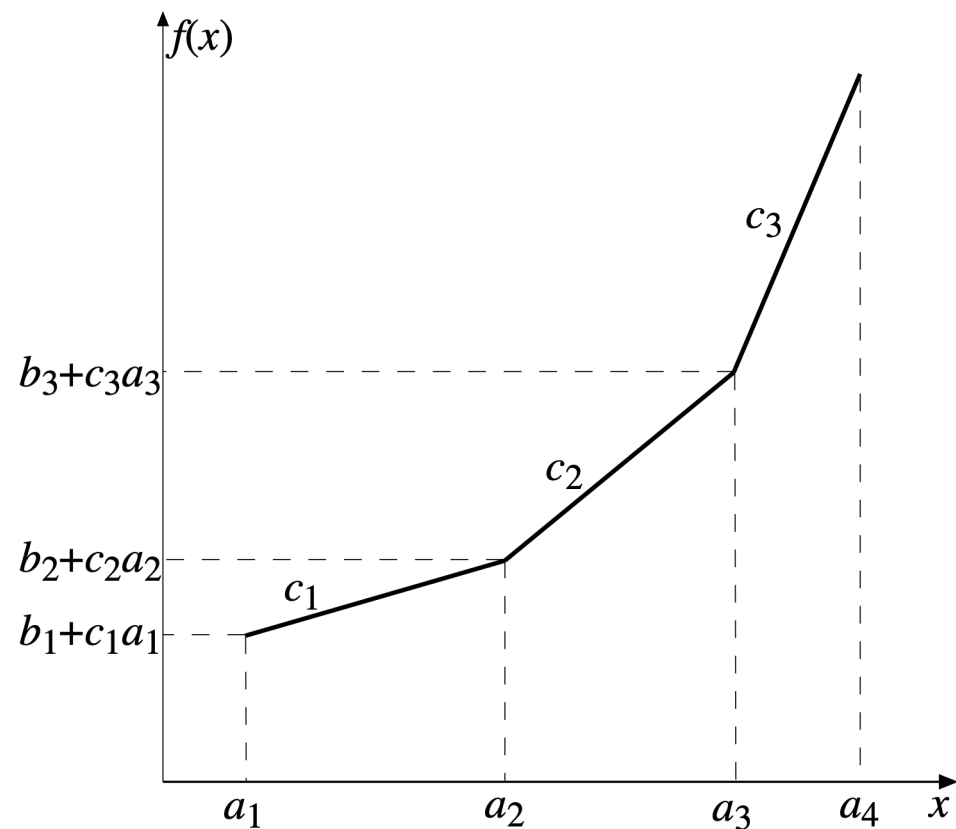
Funzioni lineari a tratti

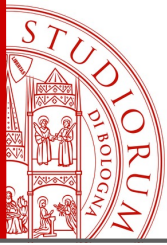
- ogni valore $x \in [a_1, a_3]$ è univocamente definito da una quadrupla (z_1, z_2, y_1, y_2)
- l'unica eccezione è $x = a_2$ che corrisponde a
- $(a_2 - a_1, 0, 1, 0)$ e $(0, 0, 0, 1)$ di cui solo il primo è corretto
- se però il problema è di min la seconda soluzione ha valore peggiore e non potrà mai essere ottima
- il meccanismo è facilmente generalizzabile al caso di più di 2 intervalli



Funzioni Convesse

- nella modellazione delle funzioni lineari a tratti la necessità di variabili ausiliarie booleane nasce dalla non convessità della funzione
- una funzione f lineare ad n tratti è **convessa** se
 - è **continua**, ossia
$$b_i + c_i a_i = b_{i+1} + c_{i+1} a_{i+1} \quad \forall i$$
 - la derivata è **non crescente**, ossia
$$c_i \leq c_{i+1} \quad \forall i$$





min Funzioni convesse

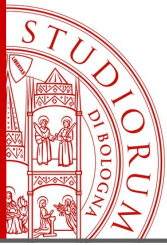
- in questo caso sono sufficienti le variabili ausiliarie quantitative z_i e il problema di minimizzazione di $f(x)$ è

$$\begin{aligned} \min & b_1 + c_1 a_1 + \sum_{i=1}^n z_i \\ & 0 \leq z_i \leq a_{i+1} - a_i \quad i = 1, \dots, n \end{aligned}$$

- se il valore ottimo è $\bar{x} = a_1 + \sum_{i=1}^n z_i$ si avrà:

$$\begin{aligned} z_i &= a_{i+1} - a_i & i &= 1, \dots, h \\ z_h &= \bar{x} - a_h & i &= h \\ z_i &= 0 & i &= h+1, \dots, n \end{aligned}$$

- analogamente per max funzioni concave



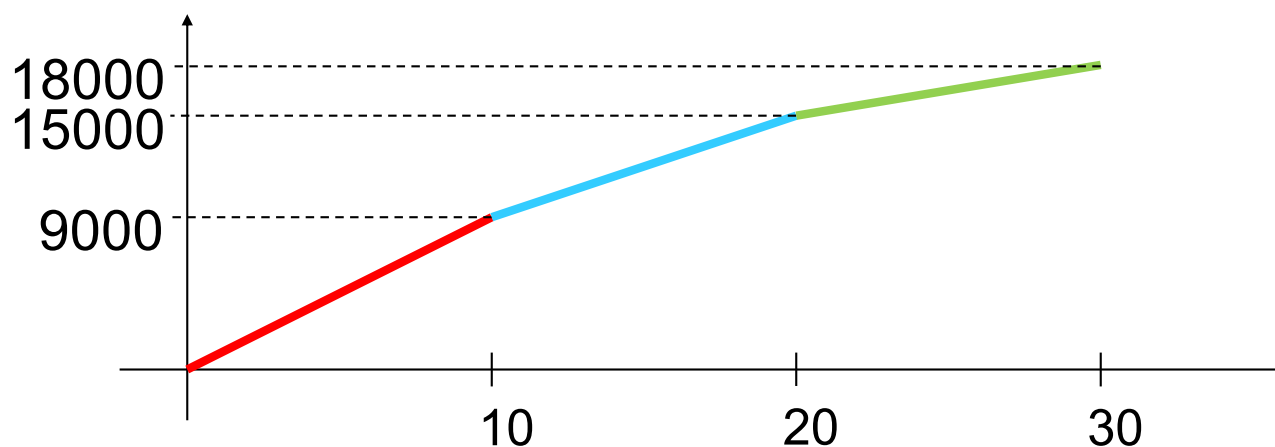
Mix di pubblicità

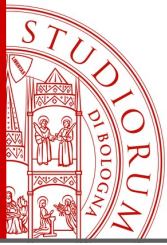
- Budget di 150 K € per pubblicizzare una nuova iniziativa.
- Due possibili canali pubblicitari:
 - Giornali: 1 K€ per annuncio
 - TV: 10 K€ per annuncio
- Al massimo 30 annunci su giornali e 15 annunci su TV
- Il numero di utenti raggiunti dipende in modo non lineare dal numero di annunci inviati.
- *massimizzare il numero totale di utenti raggiunti*

Mix di pubblicità (2)

Giornali	
n. annunci	<u>Nuovi</u> utenti per annuncio
1–10	900
11–20	600
21–30	300

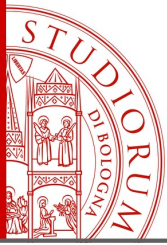
TV	
n. annunci	<u>Nuovi</u> utenti per annuncio
1–5	10000
6–10	5000
11–15	2000





Modello PLI

- si possono usare variabili binarie per indicare se le variabili decisionali sono nella 1^a, 2^a o 3^a fascia
- Vincoli di tipo logico come per il costo fisso



Modello PLI

G1, G2, G3 = n. annunci su giornali nelle 3 fasce
T1, T2, T3 = n. annunci su TV nelle 3 fasce

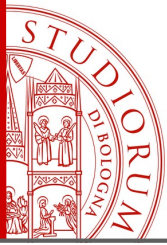
$$\max \quad 900 G1 + 600 G2 + 300 G3 + \\ 10000 T1 + 5000 T2 + 2000 T3$$

$$G1 + G2 + G3 + 10 T1 + 10 T2 + 10 T3 \leq 150$$

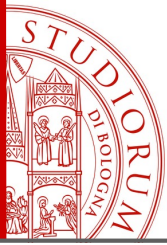
$$G1, G2, G3 \leq 10$$

$$T1, T2, T3 \leq 5$$

$$G1, G2, G3, T1, T2, T3 \geq 0, \text{ INTERE}$$



Altri modelli PLI



Variante KP: KP-bounded

- Il generico oggetto i è disponibile in p_i esemplari
- può essere scelto p_i volte

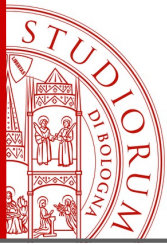
$\forall i = 1, \dots, n, x_i \geq 0$ intera = n. di copie di i scelte

$$\max c(x) = \sum_{i=1}^n c_i x_i$$

(KP - b)

$$\sum_{i=1}^n a_i x_i \leq b$$

$$0 \leq x_i \leq p_i \text{ INTERA} \quad i = 1, \dots, n$$



Variante KP: Subset-sum Problem

- Il generico oggetto i ha profitto c_i e peso $a_i = c_i$
dato un insieme di n numeri, selezionarne un sottoinsieme di somma massima e non eccedente una data soglia b
- Taglio di barre e minimizzazione scarto

$$\forall i = 1, \dots, n, \quad x_i = \begin{cases} 1 & \text{se } i \text{ selezionato} (\in S^*) \\ 0 & \text{altrimenti} \end{cases}$$

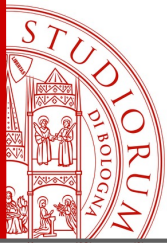
$$\max c(x) = \sum_{i=1}^n c_i x_i$$

(SSP)

$$\sum_{i=1}^n c_i x_i \leq b$$

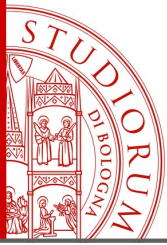
$$x_i \in \{0,1\}$$

$$i = 1, \dots, n$$



Problema KP multiplo (MKP)

- n oggetti $i = 1, \dots, n$
 - c_i "profitto" dell'oggetto i
 - a_i "peso" dell'oggetto i
- m contenitori, ciascuno di capacità b_j ($j = 1, \dots, m$)
- un oggetto può al più andare in un solo contenitore
- *impaccare nei contenitori un sottoinsieme di oggetti avente massimo profitto in modo che la somma dei pesi degli oggetti inseriti in ogni contenitore non superi la corrispondente capacità b_j*



Modello MKP

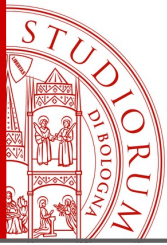
- $\forall i, j \quad x_{ij} = \begin{cases} 1 & \text{se oggetto } i \text{ inserito nel contenitore } j \\ 0 & \text{altrimenti} \end{cases}$

$$\max c(x) = \sum_{i=1}^n \sum_{j=1}^m c_i x_{ij}$$

$$\text{(MKP)} \quad \sum_{j=1}^m x_{ij} \leq 1 \quad i = 1, \dots, n$$

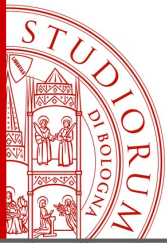
$$\sum_{i=1}^n a_i x_{ij} \leq b_j \quad j = 1, \dots, m$$

$$x_{ij} \in \{0,1\} \quad i = 1, \dots, n; j = 1, \dots, m$$



Bin Packing Problem (BPP)

- n oggetti $i = 1, \dots, n$
 - a_i "peso" dell'oggetto i
- ∞ contenitori identici, ciascuno di capacità b
 - in realtà ne servono al più n
- *impaccare tutti gli oggetti nel minor numero possibile di contenitori in modo che la somma dei pesi degli oggetti inseriti in ogni contenitore non superi la capacità b*



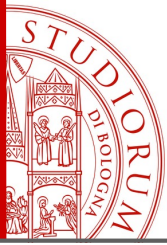
Modello PLI

$$\forall i = 1, \dots, n; j = 1, \dots, n$$

$$x_{ij} = \begin{cases} 1 & \text{se oggetto } i \text{ inserito nel contenitore } j \\ 0 & \text{altrimenti} \end{cases}$$

$$\forall j = 1, \dots, n$$

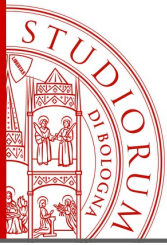
$$y_j = \begin{cases} 1 & \text{se contenitore } j \text{ utilizzato} \\ 0 & \text{altrimenti} \end{cases}$$



Modello BPP

$$\begin{aligned} & \min \sum_{j=1}^n y_j \\ & \sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, n \\ & \sum_{i=1}^n a_i x_{ij} \leq b y_j \quad j = 1, \dots, n \\ & x_{ij} \in \{0,1\} \quad i, j = 1, \dots, n \\ & y_j \in \{0,1\} \quad j = 1, \dots, n \end{aligned}$$

(BPP)



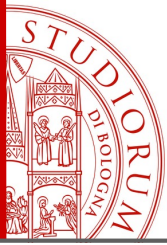
Variante: Vector Packing (VPP)

- Oltre al peso ogni oggetto i ha un volume v_i ed ogni contenitore ha anche una capacità in volume V

impaccare tutti gli oggetti nel minor numero possibile di contenitori in modo da rispettare, in ogni contenitore, sia il vincolo di peso che quello di volume

- $\sum_{i=1}^n a_i x_{ij} \leq b y_j \quad j = 1, \dots, n$
- $\sum_{i=1}^n v_i x_{ij} \leq V y_j \quad j = 1, \dots, n$

In generale: ogni oggetto i ha m dimensioni...

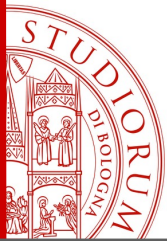


Variante: Bin con capacità variabile

- Il generico contenitore j ha capacità b_j (e costo c_j)

impaccare tutti gli oggetti in un insieme di contenitori di costo minimo in modo che la somma dei pesi degli oggetti inseriti in ogni contenitore i non superi la capacità b_j

$$\min \sum_{j=1}^n c_j y_j$$
$$\sum_{i=1}^n a_i x_{ij} \leq b_j y_j \quad j = 1, \dots, n$$



Daniele Vigo
DEI "Guglielmo Marconi"

daniele.vigo@unibo.it